

Schiavinato Sharing: BIP39-Native Threshold Backup over GF(2053) with Full Manual Fallback

Renato Schiavinato Lopez
GRIFORTIS
renato@grifortis.com

May 2026
v0.6.0*

Abstract

For long-horizon self-custody, where recovery may outlive the original software stack, hardware, and key holders, we present Schiavinato Sharing, a BIP39-native threshold-backup format that instantiates standard Shamir sharing over GF(2053). The field size is the smallest prime greater than the 2048-word BIP39 list, so each protected secret is the 1-indexed word value itself rather than an opaque derived encoding. The intended deployment is human-first and software-assisted: an offline tool performs the error-prone arithmetic, guides setup and recovery workflows, and renders artifacts, while the arithmetic layer remains implementation-independent and recoverable from durable artifacts and modular arithmetic alone. If required, share generation and recovery can also be performed fully manually.

Each share includes a position-bound linear consistency layer that deterministically catches every error pattern of up to three cells, without compromising information-theoretic confidentiality, which is preserved for any $t < k$ coalition by standard Shamir/LSSS theory. This layer detects passive arithmetic, transcription, and share-number errors; it is not an authentication mechanism against malicious share substitution.

The computational deployment profile adds a transport-integrity layer enabling per-share pre-recovery verification, mnemonic-bound and wallet-bound metadata for post-recovery error and substitution detection, and explicit lifecycle rules for temporary resume artifacts used during longer software-assisted ceremonies. Those computational and metadata mechanisms lie outside the unconditional-secrecy claim. We also characterize the bounded bias of an optional range-restricted mode intended specifically to reduce QR and hand-transcription burden.

The construction supports recursive composition for advanced multi-tier custody topologies. Versioned test vectors and in-progress prototype implementations accompany the paper.

1 Introduction

Crypto-asset self-custody often depends on a single secret—a BIP39 mnemonic [3]—serving as the master seed for hierarchical deterministic wallets [2]. Usability studies in Bitcoin key management illustrate a broader self-custody problem [10]: if this secret is lost, assets become unrecoverable; if exposed, they can be stolen. Threshold backup schemes address that single-point-of-failure problem, but long-horizon recovery introduces a second requirement: the backup format should remain recoverable even if the original software stack, device vendor, or preferred encoding disappears.

*Specification, test vectors, and in-progress prototype implementations: <https://github.com/GRIFORTIS/schiavinato-sharing>. Implementations are experimental, unaudited, and may lag the current specification. Responsible disclosure: security@grifortis.com.

1.1 Problem Setting and Deployment Thesis

This work targets *cold storage and disaster recovery*: splitting an existing BIP39 mnemonic into threshold shares stored as durable artifacts, not day-to-day key management or transaction authorization. The recovery horizon may span years or decades, during which the original software, hardware, and custodians may all become unavailable. BIP39 nativeness therefore matters: a recovered mnemonic should work directly in any BIP39-compatible wallet without format conversion or vendor-specific tooling.

The design goal is algorithmic transparency. The protocol shares the BIP39 word index directly over a prime field chosen to contain the full wordlist. There is no key derivation function (KDF), compression step, or proprietary share alphabet between the paper arithmetic and the recovered mnemonic: the manually recovered values are the BIP39 indices themselves.

The recommended deployment is human-first and software-assisted—an offline device generates shares, validates intermediate state, guides artifact production, and later assists recovery so that ordinary users do not need to perform Shamir arithmetic during normal operation. The protocol does not equate computation with permanent software dependence. We use *sovereignty* to mean that recovery does not require the original application, vendor, device, or proprietary encoding. The software-assisted deployment is the practical default for both sharing and recovery; the arithmetic layer specifies manual fallback procedures, and in the extreme a fully manual share-generation ceremony, as the continuity backstop when computation is unavailable or untrusted. The protocol is designed for both self-directed users and consultant-assisted ceremonies.

1.2 Claim Scope and Non-Goals

Schiavinato Sharing separates confidentiality, passive fault detection, and active substitution detection. The arithmetic layer is standard Shamir sharing over $GF(2053)$ applied independently to BIP39 word indices. Its confidentiality claim is information-theoretic: any coalition holding fewer than k arithmetic shares learns no information about the protected mnemonic beyond public parameters.

The row checksums, column checksums, and Global Integrity Check (GIC) form a linear consistency layer for passive faults: arithmetic mistakes, transcription errors, damaged cells, and accidental share-number mixups. This layer is not an authentication mechanism. An adversary who can rewrite a share can also recompute consistent linear checks. Active substitution is therefore addressed only by computational and operational layers: manifest audit hashes when an uncompromised manifest is available, mnemonic-bound verification after recovery, wallet-bound Recovery Verification Address (RVA) checks, artifact separation, and physical custody procedures.

The protocol does not protect against compromise of k or more shares, compromised devices or peripherals that observe secret-bearing material, physical coercion, social engineering, loss or corruption of all recovery context, or errors in the underlying BIP39/wallet ecosystem. The fully manual fallback preserves implementation-independent recovery, but it offers weaker substitution detection unless an RVA or separately protected manifest is available.

Terminology. We use *arithmetic layer* for the $GF(2053)$ Shamir construction and its row, column, and GIC checks. We use *linear consistency layer* for those row, column, and GIC checks when discussing passive-fault detection. We use *computational deployment* for the recommended offline-device profile that adds digital verification mechanisms. We use *software-assisted workflow* for the user-facing sharing and recovery flows around that deployment. The sharing workflow includes setup, resume, printing, transcription, and cleanup; the recovery workflow includes share entry or scanning, validation, interpolation, and post-recovery verification. We use *digital envelope* for the QR/Core Payload encoding around each share. We use *manual fallback* for recovery, and optionally sharing, from durable artifacts and modular arithmetic without relying

on the original software. A *resume artifact* is temporary sharing-continuity material only; it is not recovery material.

The protocol also separates computation trust from artifact-production trust. Four artifact-production tiers (Section 3, Appendix D) define how much paper output may receive secret-bearing material on each output device. In every tier, any peripheral that handles secret-bearing material is part of the trusted environment for that workflow (Section 3.3).

1.3 Contributions

Schiavinato Sharing makes the following contributions:

1. **BIP39-native threshold-backup format:** a Shamir instantiation over $\text{GF}(2053)$ —the smallest prime greater than 2048—that operates directly on 1-indexed BIP39 word indices, preserving standard BIP39 input/output without custom share alphabets or opaque seed transforms.
2. **Human-first computational deployment with full manual fallback:** the normal sharing and recovery workflows use an offline tool to reduce arithmetic and transcription burden, while the same arithmetic layer can be recovered with modular arithmetic and pre-computed Lagrange coefficients, and can be generated fully manually when computation cannot be trusted.
3. **Linear consistency layer:** a checksum architecture with provable minimum detection distance $d = 4$, enabling incremental detection of passive arithmetic and transcription errors during manual fallback workflows.
4. **Layered deployment model:** a computational deployment profile that adds a transport hash, a mnemonic-bound verification tag, wallet-bound post-recovery checks, and explicit artifact separation without folding those metadata into the information-theoretic secrecy claim.
5. **Security and reproducibility analysis:** a treatment of arithmetic-share confidentiality, Reduced Mode bias, substitution-detection limits, advanced recursive custody topologies, and version-scoped reference artifacts for independent replication.

1.4 Paper Organization

Section 2 reviews the mathematical background and situates the design relative to prior work. Section 3 defines the deployment model, artifact classes, and threat model. Section 4 specifies the arithmetic construction. Section 5 analyzes the linear consistency layer. Section 6 describes the recommended computational deployment profile. Section 7 analyzes confidentiality, Reduced Mode bias, and substitution detection. Section 8 summarizes reproducibility artifacts and preliminary evidence. Section 9 discusses limitations and future work. Section 10 concludes. Appendix A provides a worked example, Appendix B gives the payload encoding reference, Appendix C lists pre-computed Lagrange coefficients, and Appendix D summarizes deployment and artifact trust boundaries.

2 Background and Related Work

2.1 Shamir’s Secret Sharing

Shamir’s Secret Sharing (SSS) [15] splits a secret s into n shares such that any k suffice for reconstruction and any $t < k$ reveal zero information about s .

Definition 2.1.1 (Shamir Sharing over $\text{GF}(p)$). *Given secret $s \in \text{GF}(p)$, threshold k , and n share indices $x_1, \dots, x_n \in \text{GF}(p) \setminus \{0\}$:*

1. Sample $a_1, \dots, a_{k-1} \xleftarrow{\$} \text{GF}(p)$ independently and uniformly.

2. Define $f(x) = s + a_1x + \dots + a_{k-1}x^{k-1} \pmod{p}$.
3. Output shares $(x_i, f(x_i))$ for $i = 1, \dots, n$.

Recovery uses Lagrange interpolation: $s = f(0) = \sum_{j=1}^k \gamma_j \cdot y_j \pmod{p}$ where $\gamma_j = \prod_{i \neq j} \frac{x_i}{x_i - x_j} \pmod{p}$.

Security. Information-theoretic (unconditional) secrecy: for any two candidate secrets s_0, s_1 , the distribution of any $t < k$ shares is identical. This holds regardless of adversary computational resources.

2.2 Linear Secret Sharing Schemes

A Linear Secret Sharing Scheme (LSSS) [1] is one where share values are linear functions of underlying secrets. By standard theory [9], public linear relationships among shared secrets do not compromise information-theoretic security: authorized sets uniquely determine all secrets; unauthorized sets reveal zero information. This holds for arbitrary distributions on the secret space, including non-uniform distributions induced by external constraints [6].

2.3 BIP39 Mnemonics

A BIP39 mnemonic [3] encodes cryptographic entropy as a sequence of $\ell \in \{12, 15, 18, 21, 24\}$ words from a standard 2048-word list. Each word maps to an 11-bit index. For 24 words, the mnemonic encodes 256 bits of entropy plus an 8-bit SHA-256 checksum, yielding 2^{256} valid mnemonics out of $2048^{24} \approx 2^{264}$ possible sequences. An optional passphrase (“25th word”) participates in seed derivation but is external to the mnemonic encoding.

2.4 Related Work

Multi-Party Computation (MPC) wallets, multisignature schemes, and smart contract social recovery [7] address *operational security* for active asset management. Schiavinato Sharing addresses a different problem: *cold storage and disaster recovery*—how to keep threshold recovery available across long time horizons without permanently binding that recovery to one software stack. The comparisons below are therefore limited to threshold backup schemes for stored secrets.

Existing threshold backup systems vary along several axes relevant here: whether the recovered secret is native BIP39, whether the share representation admits a specified non-software recovery path, which error-detection mechanisms are available during recovery, and whether the scheme supports per-share pre-recovery verification and post-recovery mnemonic or wallet-bound verification.

Manual-capable schemes. Codex32 [8] (BIP-93) uses BCH error-correction codes and paper computation wheels (volvelles), providing strong manual error handling but requiring non-BIP39 encoding and a more specialized paper procedure. SeedXOR [14] offers a simpler mental model and standard mnemonic output, but remains inherently n -of- n and provides weak error detection. Table 1 compares these schemes with Schiavinato Sharing on the dimensions most relevant to long-horizon paper recovery.

Table 1: Selected manual-capable mnemonic-backup schemes

Feature	Schiavinato	Codex32	SeedXOR
Primary scope	BIP39	Bitcoin/BIP32 only	BIP39
Threshold flexibility	Full k -of- n	k -of- n ($k \leq 9, n \leq 31$)	n -of- n only
Manual procedure	Integer arithmetic mod 2053	BCH + volvelles/tables	XOR
Error handling	Detection ($d=4$)	Correction (BCH)	Weak/None
Per-share pre-recovery check	Yes [†]	Error detection only	No
Post-recovery verification	BIP39 + RVA [‡]	N/A (non-BIP39)	BIP39 only
BIP39 round-trip	Native I/O	No	Yes
Share representation	Paper table + optional QR	Custom bech32 strings	BIP39 words
Recursive composition	Yes	Flat	Flat

[†] Computational deployment: Transport Hash, GIC binding, and arithmetic checks validate a single share without combining shares or recovering the secret. Manifest audit hash detects substitution when a separately stored manifest is available.

[‡] The Recovery Verification Address (RVA) is a truncated wallet address recorded during setup. After recovery, the custodian derives the same address and compares; a mismatch detects substitution or context errors. RVA verification requires only offline wallet software and works in all modes, including near-fully-manual recovery. Computational deployments additionally verify the Blinded Identity (mnemonic-bound).

Software-oriented schemes. SLIP39 [16] and SSKR [17] are designed for computational recovery. Both are cryptographically sound, but the underlying arithmetic and share encodings are not specified for paper reconstruction by ordinary prime-field arithmetic. SLIP39 additionally uses a key derivation function that is not BIP39-compatible, while SSKR preserves BIP39 round-trip compatibility by operating on raw entropy and encoding shares with Bytewords/UR. Table 2 summarizes the resulting design trade-offs.

Table 2: Selected software-oriented threshold schemes

Feature	Schiavinato	SLIP39	SSKR
Arithmetic domain	Prime GF(2053)	GF(2^8) + RS1024 checksum	Extension GF(2^8)
Wallet compatibility	Native BIP39	Different derivation [†]	BIP39 round-trip [‡]
Share encoding	Table + optional QR	Custom 20-word	Bytewords/UR
Per-share pre-recovery check	Yes [§]	Share checksum only	Share checksum only
Post-recovery verification	BIP39 + BI + RVA	Reconstruction digest [#]	BIP39 checksum
Non-software recovery path	Yes (specified)	No	No
Manual arithmetic	Integer mod 2053	No	No

[†] SLIP39 uses a different key derivation function; the same entropy does not produce the same wallet as BIP39.

[‡] SSKR operates on raw BIP39 entropy and preserves the key derivation path, enabling full round-trip. Shares use the Bytewords/UR format.

[§] Transport Hash, GIC binding, arithmetic checks, and manifest audit hash enable per-share verification without combining shares or recovering the secret.

^{||} After recovery, the Blinded Identity (BI) verifies that the recovered mnemonic matches the original ($\sim 2^{96}/2^{64}$); the RVA verifies the final wallet context including derivation settings and optional passphrase ($\sim 2^{60}$ for the Bitcoin bech32/bech32m truncation analyzed here). Both are independent of the manifest.

[#] SLIP39 embeds a 4-byte digest in the sharing polynomial that verifies correct reconstruction of the master secret.

3 System Model and Threat Model

3.1 Deployment Model

Schiavinato Sharing has one arithmetic layer and several operational profiles. Full Mode is the recommended computational payload profile for long-term, high-value custody: an offline tool generates the shares, encodes the digital envelope, and renders the artifacts intended for storage. Reduced Mode retains the same software-assisted sharing and recovery workflows while

using a smaller QR payload; it is an optional manual-transcription profile for constrained-output ceremonies where QR size or hand transcription materially dominates operational risk. Its range-restriction bias is analyzed in Section 7.2. Manual recovery, and at the extreme, full manual sharing, are specified as fallback procedures for situations in which software is unavailable, incompatible, or untrusted.

Full Mode and Reduced Mode are computational payload profiles; both can produce either printed or hand-transcribed artifacts. Reduced Mode’s smaller QR grid (V3, 29×29 versus V5, 37×37) is motivated not only by payload size but by constrained-output environments—such as small-display air-gapped devices or ceremonies where manual QR transcription is required—in which a smaller grid materially reduces transcription effort and error.

This hierarchy is deliberate. Because both modes execute the same arithmetic layer, shares generated in one mode are fully usable in the other. Orthogonal to the computational payload profile, the protocol defines four artifact-production tiers ordered by output-device trust: Tier 1 (offline non-network printer, the only tier permitted to receive secret-bearing material), Tier 2 (personal network-capable printer, manifest metadata only), Tier 3 (untrusted printer, blank structure only), and Tier 4 (no printer, hand-transcription only). Full tier definitions, the artifact-class table, and the lifecycle rules for each class are given in Appendix D.

Independently of the artifact-production path, when no trusted computation device is available, the protocol falls back to fully manual arithmetic: modular arithmetic, pre-computed Lagrange coefficients, and manual transcription. This is the sovereignty-preserving continuity plan, not the recommended default. Computational deployment is operationally superior whenever a trustworthy offline tool is available, because it adds transport hashing, identity binding, and faster operator workflows.

3.2 Artifact Classes and Metadata Scope

The security claims in Schiavinato Sharing apply to different artifacts with different scopes. For clarity, we distinguish three classes:

1. **Arithmetic share layer:** the $\text{GF}(2053)$ word-share values together with the derived row checksums, column checksums, and share-bound GIC. This is the object covered by Proposition 7.1.1.
2. **Share-local recovery metadata:** human-readable hints such as label/date, derivation hint, passphrase-required indicator, and RVA. These items do not participate in the sharing arithmetic. Their role is operational and post-recovery verification. Because computational formats encode at most a coarse wallet class, deployments that rely on RVA-based verification require enough wallet-context information to be recoverable from any valid threshold set rather than only from a separate non-threshold record. In the recommended computational deployment, this means replicating the RVA together with a minimal wallet-context hint on each share artifact, while treating any manifest copy as secondary.
3. **Manifest metadata:** session- or set-level metadata such as audit hashes, optional manual inventory aids such as share-number/GIC mappings, custodian notes, and copied session identifiers. A manifest can improve logistics and pre-recovery checking, but it remains a distinct sensitive artifact and is assumed to be stored separately from all shares.

This distinction matters because manifest-aggregated data is not neutral. Computational deployments SHOULD prefer audit hashes over raw GIC maps. Raw GIC maps can help manual inventory and accidental mixup detection, but if a manifest carries at least k distinct share-bound GIC values from one session, an adversary can remove the public $+x$ term, interpolate the resulting polynomial of degree at most $k-1$, and recover $f_{\text{GIC}}(0)$, which is one linear relation of the mnemonic over $\text{GF}(2053)$. This reveals one $\text{GF}(2053)$ -valued quantity, so the leakage is upper-bounded by $\log_2(2053) \approx 11.0035$ bits; the exact reduction over the BIP39-valid mnemonic

space depends on how this relation is distributed on that domain. The leakage is small relative to BIP39 entropy, but it means such manifest data is not information-theoretically neutral and lies outside the information-theoretic secrecy claim of Proposition 7.1.1.

3.3 Threat Model

All cryptographic claims in the sequel are conditional on the following operational model.

Setup models. *Self-directed*: user generates own shares; dealer and user are identical. *Assisted*: a professional advisor provides guidance while the user maintains exclusive control of entropy and share handling.

Assumptions. Both models assume: (1) every device that handles secret-bearing material during setup or recovery—including the primary computation device and any printer, scanner, camera, or electronic calculator used in the workflow—is uncompromised and does not persist or exfiltrate sensitive data; (2) cryptographically secure random number generation; (3) accurate procedure execution; (4) shares are stored securely; and (5) if a manifest is used, it is treated as distinct sensitive metadata and stored separately, since manifest-dependent pre-recovery checks fail if the manifest is compromised. Assumption (1) applies only to peripherals that handle secret-bearing material; peripherals used exclusively for non-secret artifacts—such as manifest pages, blank templates, or audit QR codes—are not subject to this constraint (Section 6.3). A private, surveillance-resistant environment is desirable in both modes, but it is especially important for manual sharing and recovery because of the longer exposure window and the presence of physical working artifacts.

Adversary. Standard Shamir adversary obtaining fewer than k shares. Mode-specific adversaries target side-channels, malicious dependencies or firmware, and supply-chain compromise of devices or peripherals that process secret-bearing data (computational deployment), or physical observation, impression attacks, and waste recovery (manual fallback).

Out of scope. Compromise of k or more shares, physical coercion, compromised devices or peripherals that observe, persist, or exfiltrate secret-bearing data, social engineering, standard Shamir attacks [15, 18], and underlying BIP39 ecosystem vulnerabilities.

Operational mitigations. The intended deployment model favors air-gapped or minimal-purpose devices, non-networked peripherals for secret-bearing output, offline verification, explicit memory cleanup, and workflows that minimize persistent secret-bearing artifacts. These mitigations apply to secret-bearing artifacts; manifest-only metadata is treated separately in Section 7.3.

3.4 Human-First Assisted Workflows

The software-assisted workflows are the normal user experience for the computational deployment, not separate cryptographic constructions. Their purpose is to reduce human arithmetic, catch mistakes at local checkpoints, and guide artifact handling while preserving manual fallback paths. Sharing and recovery have different operational phases, but both follow the same human-first principle: software should do the arithmetic when available and trusted, while the artifacts remain sufficient for implementation-independent recovery.

During sharing, the workflow validates non-secret setup parameters, creates or validates any temporary resume material before mnemonic entry, validates the mnemonic before polynomial evaluation, computes shares and consistency checks, verifies transcription when secret-bearing fields are copied by hand, and removes temporary secret-bearing artifacts after completion.

During recovery, the workflow guides share entry or scanning, validates each share, selects a threshold set, interpolates the mnemonic, verifies row/column/GIC consistency, validates the BIP39 checksum, and performs BI/RVA checks when available.

Certain failures **MUST** stop the workflow rather than become warnings: failed authentication, malformed resume data, invalid mnemonic input, transport-hash mismatch, row/column/GIC mismatch, and runtime consistency assertion failure. Missing optional metadata such as RVA or derivation hints should be surfaced because it reduces future recovery assurance. Manual sharing and recovery remain fallback paths, not the primary user experience.

Resume artifacts. Resume artifacts are temporary sharing-continuity material only. They exist to continue an interrupted sharing ceremony; they are not recovery inputs and should not be required after shares have been produced. In the software-assisted sharing workflow, any selected resume artifact is created or authenticated before mnemonic entry, so resume continuity never requires storing the mnemonic itself. Recovery reconstructs the mnemonic from k shares by interpolation and does not consume the original resume artifact or require the original ceremony device.

Digital Resume, Hand-Transcribed Table, and Hand-Transcribed QR are sensitive while they exist. Decrypted coefficient material, or decrypted single-entropy material combined with corresponding share data, can reconstruct protected word values. After successful share creation and validation, application-managed resume material should be deleted, and externally held resume artifacts should be destroyed or separately secured. No Resume is appropriate when the sharing ceremony is expected to complete in a single session.

Hand-Transcribed QR is the only resume mode that adds a computational assumption to coefficient generation: deterministic derivation from a 256-bit single-entropy value relies on HMAC-SHA256 [12] pseudorandomness. This assumption is outside the information-theoretic secrecy claim and applies only to that resume mode. Digital Resume, Hand-Transcribed Table, and No Resume source coefficient material from true random generation.

4 Core Protocol over GF(2053)

All operational profiles share the arithmetic layer specified in this section. The output is a durable share layout whose semantics do not depend on any QR encoding or digital envelope. Later sections analyze the linear consistency properties of this layout and describe the digital envelope layered on top of it in computational deployments.

4.1 Field Selection: GF(2053)

The protocol uses GF(2053), the smallest prime field containing the 2048 BIP39 word indices. We use 1-based indexing (abandon = 1, zoo = 2048), so the protected value for word position i is the word index w_i itself. There is no hash, encrypted blob, binary seed fragment, compression step, or implementation-specific encoding between the sharing arithmetic and the recovered BIP39 mnemonic.

This choice supports implementation-independent recovery: once the field elements are reconstructed, the BIP39 word indices have been reconstructed directly. It also keeps manual fallback arithmetic in ordinary prime-field modular arithmetic, executable with a basic calculator or precomputed tables.

Because $2053 > 2048$, five field elements fall outside the BIP39 word range:

$$\{0, 2049, 2050, 2051, 2052\}.$$

These values may appear as share values, although never as recovered BIP39 word indices in a valid recovery. On share artifacts, in-range values may be displayed alongside their BIP39 words for readability; out-of-range values are represented numerically. This does not affect security or recovery correctness.

4.2 Sharing Algorithm

Parameters and notation.

- **Threshold:** (k, n) with $2 \leq k \leq n \leq 2052$ (see Section 4.4 for $k = 1$); share indices must be distinct nonzero elements of $\text{GF}(2053)$
- **Mnemonic:** $(w_1, \dots, w_\ell) \in \{1, \dots, 2048\}^\ell$ with $\ell \in \{12, 15, 18, 21, 24\}$
- **Row count:** $r = \ell/3$ (the mnemonic is arranged as r rows $\times$ 3 columns)
- **Share indices:** $x \in \{1, \dots, n\} \subset \text{GF}(2053) \setminus \{0\}$
- **Row tags:** $\tau_j^R = j$ for $j = 1, \dots, r$
- **Column tags:** $\tau_c^C \in \{10, 20, 30\}$ for $c = 1, 2, 3$
- **Row total:** $T_R = \sum_{j=1}^r j = r(r+1)/2$
- **Column total:** $T_C = 10 + 20 + 30 = 60$ (invariant across configurations)

Row and column tags serve as domain separators ensuring positional binding. Column tags are chosen from $\{10, 20, 30\}$ to be disjoint from row indices $\{1, \dots, r\}$ for all supported mnemonic lengths ($r \leq 8$), enabling unambiguous error attribution during manual verification.

Step 1: Word polynomial generation. For each word w_i , $i = 1, \dots, \ell$:

- Sample coefficients: $a_1, \dots, a_{k-1} \xleftarrow{\$} \{0, \dots, 2052\}$ independently and uniformly.
- Define $f_{w_i}(x) = w_i + a_1x + \dots + a_{k-1}x^{k-1} \pmod{2053}$.
- Evaluate: $y_{w_i}^{(x)} = f_{w_i}(x)$ for each share index x .

Step 2: Row checksum computation and incremental validation. For each row $j = 1, \dots, r$:

$$f_{R_j}(x) = (f_{w_{3j-2}}(x) + f_{w_{3j-1}}(x) + f_{w_{3j}}(x)) + \tau_j^R \pmod{2053} \quad (1)$$

where $\tau_j^R = j$ is added to the constant term only. No additional randomness is required. At any share index x :

$$y_{R_j}^{(x)} = (y_{w_{3j-2}}^{(x)} + y_{w_{3j-1}}^{(x)} + y_{w_{3j}}^{(x)} + j) \pmod{2053} \quad (2)$$

This row-level structure enables *incremental validation* during share construction: after computing every three word shares and their row checksum, each share's row values can be immediately verified via Equation (2), catching arithmetic or transcription errors before proceeding to the next row.

Step 3: Column checksum computation. For each column $c = 1, 2, 3$:

$$f_{C_c}(x) = \sum_{\substack{i: \text{word } i \\ \text{is in column } c}} f_{w_i}(x) + \tau_c^C \pmod{2053} \quad (3)$$

At any share index x :

$$y_{C_c}^{(x)} = \left(\sum_{i \in \text{col } c} y_{w_i}^{(x)} + \tau_c^C \right) \pmod{2053} \quad (4)$$

Step 4: Global Integrity Check (GIC). Define the base GIC polynomial by summing all word polynomials and adding the row and column totals to the constant term (enabling addition-only validation paths):

$$f_{\text{GIC}}(x) = \sum_{i=1}^{\ell} f_{w_i}(x) + T_R + T_C \pmod{2053} \quad (5)$$

The *share-bound* GIC on each share adds the share number for index binding:

$$\text{GIC}(x) = (f_{\text{GIC}}(x) + x) \bmod 2053 \quad (6)$$

Three equivalent validation paths hold at any share index x :

$$\text{GIC}(x) \equiv \sum_{i=1}^{\ell} y_{w_i}^{(x)} + T_R + T_C + x \pmod{2053} \quad (7)$$

$$\equiv \sum_{j=1}^r y_{R_j}^{(x)} + T_C + x \pmod{2053} \quad (8)$$

$$\equiv \sum_{c=1}^3 y_{C_c}^{(x)} + T_R + x \pmod{2053} \quad (9)$$

Authentication boundary. The row checksums, column checksums, and GIC provide passive-fault detection, not authenticity. An adversary who can rewrite a share can recompute consistent linear checks. Computational deployments therefore add separate verification mechanisms: manifest Audit Hashes for pre-recovery substitution detection when an uncompromised manifest is available, and BI/RVA checks for post-recovery mnemonic- and wallet-bound verification.

Step 5: Share assembly. Each share at index x contains $\ell + r + 3 + 1$ field elements:

$$S_x = (y_{w_1}^{(x)}, \dots, y_{w_\ell}^{(x)}, y_{R_1}^{(x)}, \dots, y_{R_r}^{(x)}, y_{C_1}^{(x)}, y_{C_2}^{(x)}, y_{C_3}^{(x)}, \text{GIC}(x))$$

For a 24-word mnemonic: $24 + 8 + 3 + 1 = 36$ values per share.

BIP39 passphrase independence. The BIP39 passphrase is external to the sharing arithmetic and is never encoded in share payloads. Users relying on a passphrase must back it up separately.

4.3 Recovery Algorithm

Input. k shares $S_{x_1}, \dots, S_{x_k}$.

Step 1: Per-share validation. For each share at index x , verify the share-bound GIC using any path from Equations (7)–(9). The $+x$ term binds the GIC to the share number: if x is mislabeled, validation fails.

Step 2: Lagrange coefficient computation and sanity check. Compute the interpolation coefficients

$$\gamma_j = \prod_{i \neq j} \frac{x_i}{x_i - x_j} \pmod{2053}, \quad j = 1, \dots, k.$$

Then verify:

$$\sum_{j=1}^k \gamma_j \cdot x_j \equiv 0 \pmod{2053} \quad (10)$$

This holds because Lagrange interpolation of $g(x) = x$ at $x = 0$ yields $g(0) = 0$. Failure indicates coefficient error.

Lagrange coefficients are determined entirely by share indices and contain zero secret information. They may be pre-computed for common schemes and published (Appendix C), or computed on any device—even untrusted—without leaking secrets.

Step 3: Row-by-row interpolation with incremental validation. For each row $j = 1, \dots, r$, interpolate the three word values and the row checksum:

$$\hat{s} = \sum_{m=1}^k \gamma_m \cdot y_s^{(x_m)} \pmod{2053}$$

for $s \in \{w_{3j-2}, w_{3j-1}, w_{3j}, R_j\}$. Immediately verify:

$$(\hat{w}_{3j-2} + \hat{w}_{3j-1} + \hat{w}_{3j} + j) \pmod{2053} \stackrel{?}{=} \hat{R}_j$$

A mismatch localizes the error to the current row, allowing correction before proceeding. This incremental structure is the primary error-catching mechanism during manual recovery.

Step 4: Global validation. After all rows pass, interpolate the three column checksums $\hat{C}_1, \hat{C}_2, \hat{C}_3$ and verify each via Equation (4). Verify the recovered GIC via Equations (7)–(9).

The $+x$ terms in the share-bound GIC cancel automatically during interpolation by Equation (10). The row tags τ_j^R and column tags τ_c^C do *not* cancel, as they are embedded in the constant terms of the checksum polynomials.

Optional GIC cross-check. If share-bound GIC values are available from all k shares, they may be independently interpolated at $x = 0$. The $+x$ terms cancel, yielding $f_{\text{GIC}}(0) = \sum_{i=1}^{\ell} w_i + T_R + T_C$. Comparing this against the same sum computed from recovered words provides an independent validation path that does not depend on row or column checksums.

Step 5: Output. Convert recovered word indices $(\hat{w}_1, \dots, \hat{w}_{\ell})$ to BIP39 words using the 1-indexed wordlist.

4.4 Advanced Custody Topologies: Recursive Composition

The construction supports recursive composition for advanced custody topologies: a completed share from layer L becomes the protected object of layer $L + 1$. This profile is intended for institution-scale or high-value custody structures where software handles artifact tracking, validation, and recovery guidance. It is not the recommended baseline for ordinary individual backups, because each layer increases ceremony complexity, metadata management, and recovery coordination risk. Each layer remains standard Shamir over GF(2053) with independently computed row checksums, column checksums, and GIC.

Intermediate-layer semantics. At layer $L > 0$, the “word” values being shared are GF(2053) elements from the parent share, not BIP39 word indices. Values $\{0, 2049, 2050, 2051, 2052\}$ are valid inputs. BIP39 validity is checked *only* at the final layer-0 recovery. Identity binding (Section 6.2) applies only at the outermost layer.

Degenerate case: $k = 1$. When $k = 1$, the polynomial is constant ($f(x) = a_0$) and all shares are identical copies of the parent value. This provides replication for access-control grouping (e.g., “any member of this group can represent the group”) without threshold protection at that layer.

Depth limits. The sharing arithmetic itself imposes no nesting limit—any share can be recursively shared indefinitely. The practical depth constraint comes from the digital envelope, which allocates a fixed number of bits for per-layer threshold and index metadata (Section 6.1): Full Mode supports up to 4 active layers; Reduced Mode supports up to 2. Manual fallback ceremonies without a digital envelope are not subject to this limit.

5 Linear Consistency Layer

The share layout is intentionally redundant. Its row checksums, column checksums, and share-bound GIC form a linear code over $\text{GF}(2053)$ that supports incremental validation during manual fallback workflows. This section analyzes that redundancy as part of the arithmetic share layer; it is not an authentication mechanism against deliberate substitution.

5.1 Error-Detection Properties

The share table for an ℓ -word mnemonic contains $N = \ell + r + 3 + 1$ cells, where $r = \ell/3$. The $r + 4$ check equations (r row, 3 column, 1 GIC) define a linear code over $\text{GF}(2053)$.

Definition 5.1.1 (Check equations). *An error vector*

$$\mathbf{e} = (e_{w_1}, \dots, e_{w_\ell}, e_{R_1}, \dots, e_{R_r}, e_{C_1}, e_{C_2}, e_{C_3}, e_{GIC})$$

is undetectable if it satisfies all $r + 4$ check equations:

$$e_{w_{3j-2}} + e_{w_{3j-1}} + e_{w_{3j}} = e_{R_j} \quad \text{for } j = 1, \dots, r \quad (11)$$

$$\sum_{i \in \text{col } c} e_{w_i} = e_{C_c} \quad \text{for } c = 1, 2, 3 \quad (12)$$

$$\sum_{i=1}^{\ell} e_{w_i} = e_{GIC} \quad (13)$$

Theorem 5.1.2 (Independence and rank). *The $r + 4$ check equations are linearly independent over $\text{GF}(2053)^N$. The check matrix has rank $r + 4$.*

Proof. Each row check equation (11) contains a unique variable e_{R_j} appearing in no other equation ($j = 1, \dots, r$). Each column check equation (12) contains a unique variable e_{C_c} ($c = 1, 2, 3$). The GIC equation (13) contains a unique variable e_{GIC} . A system of $r + 4$ equations where each has at least one variable exclusive to it is linearly independent. $\square$

Parity-check matrix. Let $H \in \text{GF}(2053)^{(r+4) \times N}$ be the parity-check matrix whose rows are indexed by $R_1, \dots, R_r, C_1, C_2, C_3, G$. Let $\mathbf{u}_{R_j}, \mathbf{u}_{C_c}, \mathbf{u}_G$ denote the corresponding standard basis vectors in $\text{GF}(2053)^{r+4}$. The columns of H are

$$h_{j,c} = \mathbf{u}_{R_j} + \mathbf{u}_{C_c} + \mathbf{u}_G, \quad h_{R_j} = -\mathbf{u}_{R_j}, \quad h_{C_c} = -\mathbf{u}_{C_c}, \quad h_G = -\mathbf{u}_G.$$

Here $h_{j,c}$ denotes the column corresponding to the word cell in row j and column c . By Definition 5.1.1, an error vector is undetectable if and only if it lies in $\ker H$.

Lemma 5.1.3 (Distance and dependent columns). *For a linear code $C = \ker H$, the minimum Hamming weight of a nonzero codeword equals the minimum size of a linearly dependent set of columns of H .*

Proof. If $\mathbf{z} \in C$ is a nonzero codeword with support S , then $H\mathbf{z}^T = 0$ gives a nontrivial linear relation among the columns indexed by S , so those columns are linearly dependent. Conversely, if a set of columns indexed by S is linearly dependent, a nontrivial relation among them produces a nonzero vector $\mathbf{z} \in \ker H$ supported on S . Thus the smallest nonzero codeword weight equals the smallest dependent-column set size. $\square$

Theorem 5.1.4 (Minimum detection distance). *The minimum weight of a nonzero undetectable error vector is exactly $d = 4$. That is, all error patterns affecting 1, 2, or 3 cells are detected with certainty.*

Proof. By Lemma 5.1.3, it suffices to show that every set of 1, 2, or 3 columns of H is linearly independent, and that some set of 4 columns is linearly dependent.

One column. No column of H is zero, so no one-column dependence exists.

Two columns. If two nonzero columns are linearly dependent, one must be a nonzero scalar multiple of the other, hence they must have the same support. But all columns of H have distinct supports:

$$\text{supp}(h_{j,c}) = \{R_j, C_c, G\}, \quad \text{supp}(h_{R_j}) = \{R_j\}, \quad \text{supp}(h_{C_c}) = \{C_c\}, \quad \text{supp}(h_G) = \{G\}.$$

Therefore every two-column set is linearly independent.

Three columns. Let m be the number of word columns in the set.

1. If $m = 0$, the columns are distinct signed basis vectors and are linearly independent.
2. If $m = 1$, say the set contains $h_{j,c}$, then any nontrivial dependence must cancel the coordinates G , R_j , and C_c . Among check columns, only h_G can cancel G , only h_{R_j} can cancel R_j , and only h_{C_c} can cancel C_c . Thus a dependence involving $h_{j,c}$ requires all three of those check columns in addition to $h_{j,c}$, for a total of 4 columns.
3. If $m = 2$, say the set contains h_{j_1,c_1} and h_{j_2,c_2} , then the G coordinate forces their coefficients to sum to zero, so their contribution is a nonzero scalar multiple of

$$h_{j_1,c_1} - h_{j_2,c_2}.$$

If the two word columns share neither row nor column, this difference has four nonzero row/column coordinates, which one check column cannot cancel. If they share a row, the difference equals $\mathbf{u}_{C_{c_1}} - \mathbf{u}_{C_{c_2}}$ and requires two column-check columns. If they share a column, it requires two row-check columns. Hence no three-column dependence exists with $m = 2$.

4. If $m = 3$, suppose there is a nontrivial dependence among three word columns. If any coefficient were zero, this would reduce to a one- or two-column dependence, already excluded. So all three coefficients are nonzero. View the three word columns as edges in a bipartite graph whose left vertices are rows and whose right vertices are columns. The row and column coordinates of the dependence imply that, at every incident vertex, the sum of coefficients on incident edges is zero. Therefore no incident vertex can have degree 1, since then the unique incident coefficient would be forced to zero. But every finite acyclic graph has a vertex of degree 1, so the selected three-edge subgraph must contain a cycle. A bipartite cycle has even length, hence at least 4, impossible with only 3 edges. Thus no three-word dependence exists.

Four columns (achievable). Dependent four-column sets exist, for example:

$$\begin{aligned} h_{j,c} + h_{R_j} + h_{C_c} + h_G &= 0, \\ h_{j_1,c_1} - h_{j_1,c_2} - h_{j_2,c_1} + h_{j_2,c_2} &= 0, \\ h_{j,c_1} - h_{j,c_2} + h_{C_{c_1}} - h_{C_{c_2}} &= 0, \\ h_{j_1,c} - h_{j_2,c} + h_{R_{j_1}} - h_{R_{j_2}} &= 0. \end{aligned}$$

These correspond respectively to the point, rectangle, row-pair, and column-pair undetectable error patterns.

Hence the smallest dependent column set has size 4, so the minimum weight of a nonzero undetectable error vector is exactly $d = 4$. $\square$

Corollary 5.1.5 (Undetected-error probability). *Under a uniform random error model over $\text{GF}(2053)^N$, the probability that a nonzero error is undetectable is $1/2053^{r+4}$ for a mnemonic of ℓ words ($r = \ell/3$). Concrete values:*

ℓ	r	Checks ($r+4$)	$P_{undetected}$
12	4	8	$1/2053^8 \approx 3.2 \times 10^{-27}$
15	5	9	$1/2053^9 \approx 1.5 \times 10^{-30}$
18	6	10	$1/2053^{10} \approx 7.5 \times 10^{-34}$
21	7	11	$1/2053^{11} \approx 3.7 \times 10^{-37}$
24	8	12	$1/2053^{12} \approx 1.8 \times 10^{-40}$

Proof. By Theorem 5.1.2, the null space of the check matrix has dimension $N - (r + 4)$. The fraction of nonzero vectors in the null space is $(2053^{N-(r+4)} - 1)/(2053^N - 1) \approx 1/2053^{r+4}$. $\square$

Practical note. Under realistic (non-uniform) error models, the deterministic detection of all weight- ≤ 3 errors (Theorem 5.1.4) is the primary assurance for manual use; the $1/2053^{r+4}$ bound applies to worst-case uniform random corruption.

Appendix A illustrates the full workflow on a toy instance over GF(13).

6 Computational Deployment and Verification

The arithmetic share layer is format-agnostic, but the recommended deployment wraps each share in a digital envelope for QR transport and stronger verification. The reference byte-level encoding is specified in Appendix B. The mechanisms in this section are computational checks layered on top of the arithmetic share layer; they are not part of the information-theoretic secrecy claim of Proposition 7.1.1.

6.1 Computational Payload Profiles and Manual-Transcription Trade-Offs

Two computational payload profiles balance verification strength, payload size, and manual transcribability. Full Mode is the default for long-term, high-value custody. Reduced Mode is an optional profile for constrained-output ceremonies where QR size or hand transcription materially dominates operational risk:

Table 3: Computational deployment profiles

Property	Full Mode	Reduced Mode
Word-share range	Full GF(2053)	11-bit word shares [†]
QR footprint	Larger	Smaller
Pre-recovery transport check	Yes	QR/LCL only
Mnemonic-bound check	$\sim 2^{96}$	$\sim 2^{64}$
Nesting metadata	Up to 4 layers	Up to 2 layers
Range-restriction bias	None	Small; bounded in Section 7.2

[†] Share-bound GIC remains in GF(2053). Word polynomials producing any share value > 2047 are rejected and regenerated.

Reduced Mode intentionally conditions polynomial selection on compact serialized range. It is therefore not part of the exact perfect-secrecy claim in Proposition 7.1.1. Its role is operational: reducing QR size and hand-transcription burden when that reduction materially improves the ceremony. Full Mode remains the default recommendation for long-term, high-value custody.

6.2 Verification Layers

Transport Hash (Full Mode). Full Mode includes a truncated SHA-256 [11] hash over the serialized Core Payload. This is a transport-integrity check for random bit flips, scanning errors, or physical damage before interpolation. It is a computational check on the digital envelope, not part of the arithmetic secrecy claim.

Mnemonic-bound verification (BI). Computational shares also carry a session identifier and a Blinded Identity (BI). The BI is an HMAC-SHA256 [12] tag keyed by a mnemonic-derived master-key identifier and evaluated on the session identifier. After recovery, software recomputes the tag from the recovered mnemonic; a mismatch indicates share mixing, corruption, or substitution. The mnemonic-derived key is internal and is not serialized on shares. For an independently wrong mnemonic, accidental BI matching is about 2^{-96} in Full Mode and 2^{-64} in Reduced Mode.

Wallet-bound verification (RVA). The BI is mnemonic-bound; it does not by itself verify wallet context, derivation settings, or BIP39 passphrase use. As a complementary check, the protocol allows recording a Recovery Verification Address (RVA): a truncated address derived from the intended target wallet and compared after recovery. For Bitcoin bech32/bech32m addresses [4, 5], the “first 8 characters + last 8 characters” form gives about 2^{60} matching strength under the address-format assumptions analyzed here. Other asset ecosystems require format-specific matching and privacy analysis. If RVA-based verification is relied upon, enough wallet-context information must be recoverable from any valid threshold set; in the recommended computational deployment, the RVA and a minimal wallet-context hint are therefore replicated on each share artifact, while any manifest copy is secondary.

6.3 Manifest-Based Pre-Recovery Audit

An optional manifest can store session metadata and per-share audit hashes without containing plaintext secret shares. When the manifest is stored separately from the shares, these hashes enable pre-recovery audit of a single share: a trusted device scans or imports the share, validates the available local checks, recomputes the Core Payload hash, and compares it with the manifest record. This detects active share substitution when the manifest remains uncompromised.

The audit hash commits to the Core Payload bytes: header fields, session metadata, serialized word-share values, and printed GIC. Row and column checksum cells are not serialized in the Core Payload and are therefore not independently committed by this hash; they are deterministic functions of the word-share values and must be recomputed during scanning, re-entry, or manual validation. Manifest-based audit is conditional on artifact separation: an adversary who can alter both a share and its manifest record can defeat the pre-recovery audit.

7 Security Analysis

This section separates three kinds of claims: information-theoretic secrecy of the arithmetic share layer, the small bounded deviation introduced by Reduced Mode, and substitution-detection properties that depend on operational artifacts or computational checks.

7.1 Confidentiality

Proposition 7.1.1 (Information-theoretic confidentiality of the arithmetic share layer). *For any $t < k$ and any two BIP39-valid mnemonics M_0, M_1 , the distribution of any t arithmetic shares is identical, where an arithmetic share consists of the GF(2053) word-share values together with the derived checksum fields (row checksums, column checksums, and GIC).*

Proof sketch. The ℓ word indices are protected by independent Shamir polynomials of degree at most $k-1$ with uniformly random coefficients over GF(2053). By Shamir’s theorem [15], the distribution of $t < k$ word-share values is identical for any constant term. The checksum fields (row checksums, column checksums, and GIC) are deterministic affine functions of the word shares with only public coefficients and public offsets $(\tau_j^R, \tau_c^C, T_R, T_C, x)$, adding zero information to the adversary’s view. By LSSS theory [1, 9], restricting the secret domain to BIP39-valid mnemonics does not change this: perfect secrecy is distribution-agnostic. $\square$

Scope. Proposition 7.1.1 applies only to the unrestricted arithmetic share layer, including Full Mode and manual operation. Reduced Mode is an optional manual-transcription profile and intentionally departs from exact perfect secrecy because rejection sampling conditions the word polynomials on the serialized range restriction; this deviation is analyzed separately in Section 7.2. Optional authenticity metadata such as the Blinded Identity and Recovery Verification Address strengthen substitution detection, but they are verification tags and are not covered by this information-theoretic secrecy claim.

Proposition 7.1.2 (Entropy preservation). *Effective keyspace remains $\approx 2^{256}$ for 24-word mnemonics. Sharing over $\text{GF}(2053)$ introduces no compression.*

Justification. BIP39 entropy for $\ell = 24$ is 256 bits, determining 2^{256} valid mnemonics. Each word index $w_i \in \{1, \dots, 2048\} \subset \text{GF}(2053)$ is mapped injectively into the field without compression. The ℓ independent Shamir polynomials operate on these indices word by word; no entropy is combined, truncated, or hashed during sharing. The linear consistency fields are deterministic linear functions of the word shares and add no entropy of their own. $\square$

Note on the coupled-constraint question. A previous version of this work [13] posed confidentiality under coupled BIP39/linear constraints as an open question. We observe that this follows directly from LSSS theory: Shamir’s perfect secrecy is distribution-agnostic, and the linear consistency fields add no information beyond the t word share values already provided. We invite independent formal verification.

7.2 Reduced Mode Range-Restriction Bias

Reduced Mode exists to reduce QR size and hand-transcription burden in constrained-output ceremonies. It rejects word polynomials if any share evaluation exceeds 2047. This introduces a small bias because accepted polynomials are sampled from a restricted subset of the Shamir polynomial space. Rather than estimate this effect from a uniqueness or independence heuristic, we give a conservative bound that applies to any unauthorized set $t < k$.

Lemma 7.2.1 (Uniformity at one unseen position). *Fix distinct points $0, x_1, \dots, x_t, x^* \in \text{GF}(2053)$ with $t < k - 1$, and fix values $s, y_1, \dots, y_t \in \text{GF}(2053)$. Let $\mathcal{A}$ be the affine space of polynomials f of degree at most $k - 1$ satisfying*

$$f(0) = s, \quad f(x_i) = y_i \quad \text{for } i = 1, \dots, t.$$

Then the evaluation map

$$\text{ev}_{x^*} : \mathcal{A} \rightarrow \text{GF}(2053), \quad f \mapsto f(x^*)$$

is surjective, and every value in $\text{GF}(2053)$ has exactly $|\mathcal{A}|/2053$ preimages. Consequently, for any subset $F \subset \text{GF}(2053)$, the fraction of polynomials in $\mathcal{A}$ with $f(x^) \in F$ is exactly $|F|/2053$.*

Proof. Define

$$g(X) = X \prod_{i=1}^t (X - x_i).$$

Since $t < k - 1$, we have $\deg g = t + 1 \leq k - 1$. Also, $g(0) = 0$ and $g(x_i) = 0$ for all $i = 1, \dots, t$, while $g(x^*) \neq 0$ because x^* is distinct from $0, x_1, \dots, x_t$.

Let $f \in \mathcal{A}$ and let $z \in \text{GF}(2053)$ be arbitrary. Setting

$$\lambda = \frac{z - f(x^*)}{g(x^*)}$$

gives $f + \lambda g \in \mathcal{A}$ and $(f + \lambda g)(x^*) = z$. Thus ev_{x^*} is surjective.

Its fibers are translates of $\ker(\text{ev}_{x^*})$, so all fibers have equal size. Since the codomain has size 2053, each value has exactly $|\mathcal{A}|/2053$ preimages. The final statement follows immediately. $\square$

Proposition 7.2.2 (Conservative Reduced Mode bias bound). *Fix one word polynomial in an (k, n) scheme and an adversary holding $t < k$ shares. Let $m = n - t$ be the number of unseen share positions. Then the Reduced Mode range restriction excludes at most a $5m/2053$ fraction of otherwise compatible candidates. Equivalently, the surviving fraction is at least*

$$1 - \frac{5m}{2053} = \frac{2053 - 5m}{2053}.$$

For an ℓ -word mnemonic, this yields a conservative candidate-space reduction bound of at most

$$\ell \cdot \log_2 \left(\frac{2053}{2053 - 5m} \right)$$

bits.

This is negligible relative to the 128–256 bits of entropy represented by standard BIP39 mnemonic lengths. The point of the bound is not that Reduced Mode is perfectly uniform, but that the deliberate usability trade-off is quantified and remains far below the security margin of the protected mnemonic space.

Proof sketch. The forbidden set for a Reduced Mode word share is

$$F = \{2048, 2049, 2050, 2051, 2052\},$$

which has size 5.

If $t = k - 1$, then each candidate secret determines a unique polynomial. At any unseen share index x , Lagrange interpolation with nodes $\{0, x_1, \dots, x_{k-1}\}$ gives

$$f(x) = \alpha_x s + \beta_x$$

for constants $\alpha_x, \beta_x \in \text{GF}(2053)$ determined by the observed shares, with

$$\alpha_x = \prod_{i=1}^{k-1} \frac{x - x_i}{-x_i} \neq 0$$

because $x \notin \{0, x_1, \dots, x_{k-1}\}$. Hence $s \mapsto f(x)$ is an affine bijection, so at most 5 candidate secrets are rejected at that position. By union bound over the m unseen positions, at most $5m$ candidate secrets are rejected in total.

If $t < k - 1$, then after fixing the candidate secret and the observed shares, the compatible polynomials form an affine space of positive dimension. By Lemma 7.2.1, for any unseen share index x , exactly a $5/2053$ fraction of these compatible polynomials satisfy $f(x) \in F$. Hence the rejection fraction per unseen position is exactly $5/2053$, and by union bound the total rejection fraction is at most $5m/2053$.

No independence assumption is required. □

Concrete 24-word bounds. For the configurations emphasized in this work, the conservative 24-word bounds are:

$$\begin{aligned} \text{2-of-3 with } t = 1 \ (m = 2) : & \leq 0.169 \text{ bits,} \\ \text{2-of-4 with } t = 1 \ (m = 3) : & \leq 0.255 \text{ bits,} \\ \text{3-of-5 with } t = 2 \ (m = 3) : & \leq 0.255 \text{ bits.} \end{aligned}$$

Across all unauthorized sets of these same topologies, the worst case is $m = 5$, giving

$$24 \cdot \log_2 \left(\frac{2053}{2028} \right) \approx 0.424$$

bits.

7.3 Substitution Detection

Linear consistency layer: zero substitution detection. Following the cheater framework of Tompa and Woll [19], an adversary with physical access to one share can choose arbitrary word values and compute consistent row checksums, column checksums, and GIC. The modified share passes all $r + 4$ check equations. After Lagrange interpolation with the fake share and $k - 1$ genuine shares, the recovered result is internally consistent (linearity is preserved) but produces a wrong mnemonic. No per-share inspection can distinguish a substituted share from a genuine one.

Pre-recovery detection (manifest-dependent). If an uncompromised manifest is available, certain checks can be performed *before* recovery on a per-share basis:

1. **Share-bound GIC comparison** (all modes): compare the share’s GIC against the manifest entry for that share number. This detects accidental corruption or cross-session share mixing, but *not* deliberate substitution—an adversary can choose word values that produce the same GIC by preserving the word sum modulo 2053.
2. **Audit Hash** (computational deployment): recompute SHA-256(Core Payload) from the scanned/imported share and compare against the manifest record. This detects *both* accidental and deliberate substitution, since the adversary cannot produce a different payload with the same hash (SHA-256 collision resistance).

Both checks are conditional: if the adversary also compromises the manifest, pre-recovery detection fails. Manifest content—Session Batch ID, Blinded Identity, share indices, optional manifest GIC values, and audit hashes—contains no plaintext secret shares, but it is not information-theoretically neutral: the Blinded Identity is a mnemonic-dependent verification tag, and k distinct manifest GIC values reveal one linear relation of the mnemonic (Section 3.2). Although this metadata does not by itself reveal the mnemonic, it remains operationally sensitive: collected manifest artifacts can assist targeting, share harvesting, or recovery sabotage. Conservative computational deployments should prefer audit hashes over raw GIC maps, use trusted peripherals when practical, and continue to store manifest artifacts separately from all shares. The audit hash provides a verification oracle for candidate secrets, but the search space remains the full mnemonic keyspace (2^{128} to 2^{256}) because SHA-256 covers all word values jointly and does not enable per-word testing.

Post-recovery detection (manifest-independent). After Lagrange interpolation produces a candidate mnemonic, several checks can reject wrong recoveries without relying on a manifest:

1. **BIP39 checksum** ($\sim 99.6\%$ for 24 words): an independent non-linear check on the recovered mnemonic. It detects many wrong recoveries, but it is not an authentication tag.
2. **Identity Lock** (computational deployment): because the Session Batch ID and Blinded Identity are present in each share payload, the check is available even when the manifest is lost. For an independently wrong recovered mnemonic, the Blinded Identity matches accidentally with probability about 2^{-96} in Full Mode or 2^{-64} in Reduced Mode.
3. **RVA** (all modes): for the Bitcoin bech32/bech32m truncation analyzed in Section 6.2, an independently wrong recovered wallet context matches with probability about 2^{-60} . Other address formats require their own matching-strength analysis.

These checks require only the recovered mnemonic and recorded share-local metadata. They are post-recovery filters, not proof that the input shares were authentic.

In manual fallback without RVA, post-recovery substitution detection relies solely on BIP39 checksum and operational mitigations (tamper-evident storage, distributed manifests).

8 Evaluation and Reproducibility

This section reports the current reproducibility artifacts and preliminary empirical evidence. The v0.6.0 test vectors are intended to support independent verification of the protocol. Prototype software exists to support experimentation and implementation work, but it should not be treated as a conformant reference for the current specification unless explicitly versioned as such. The timing and usability observations are descriptive, author-generated measurements rather than controlled studies.

Prototype implementations. Prototype implementations are in progress in three forms:

- JavaScript/TypeScript library (@grifortis/schiavinato-sharing)
- Python library
- Offline single-file HTML tool for air-gapped environments

All implementations are open-source (MIT), experimental, unaudited, and intended for learning, replication, and review rather than operational deployment. Some prototype implementations may lag this whitepaper and the current test vectors.

Test vectors. Version-scoped test vectors in `test_vectors/` provide reproducible fixtures for independent checking, including polynomial coefficients, share values, and expected checksums.

Preliminary manual timing benchmarks (author-measured, v0.6.0). Measured under the current protocol (position-bound row checksums, column checksums, GIC with row/column totals):

Configuration	Manual sharing	Manual recovery
12-word, 2-of-3	~2 hours	~30 minutes
24-word, 3-of-5	~5 hours	~60 minutes

Manual sharing includes entropy generation, polynomial evaluation, all checksum computations, share transcription, and final validation. Manual recovery includes per-share GIC validation, Lagrange interpolation (using pre-computed coefficient tables), row/column/GIC cross-checks, and BIP39 word conversion.

Manual entropy generation. The Schiavinato RNG uses a $7 \times 7 \times 7 \times 6 = 2058$ rejection sampling scheme. A tuple (a, b, c, d) with $a, b, c \in \{0, \dots, 6\}$, $d \in \{0, \dots, 5\}$ maps to $t = (((a \cdot 7 + b) \cdot 7 + c) \cdot 6 + d) \in \{0, \dots, 2057\}$. Reject $t \geq 2053$ (probability $5/2058 \approx 0.24\%$). In timed runs, coefficient generation averaged ~45 seconds each, yielding ~36 minutes for the 48 coefficients required by a 24-word, 3-of-5 configuration.

Usability context. An informal pilot with four non-technical participants (ages 28–72) on an early prototype suggested that (i) procedural retention was poor after 13 months, indicating that self-documenting materials and checkpointed workflows are important for inheritance scenarios, and (ii) manual Lagrange computation was the primary bottleneck, motivating the pre-computed coefficient tables in the current protocol. This pilot should be interpreted as hypothesis-generating only; it pre-dated the current checksum architecture and tested a harder configuration (4-of-16), so it is not representative of v0.6.0 performance.

9 Discussion

The current manuscript should be read as an exploratory research artifact rather than as an audited operational system.

9.1 Limitations

- **No strong manual identity binding in the current design:** the purely manual GF(2053) checks in Schiavinato are low-degree relations intended for error detection, not high-entropy authenticity tags. In the current protocol they do not provide meaningful mnemonic- or wallet-bound identity binding; the RVA addresses this but requires a one-time computational step during the sharing ceremony.
- **The linear consistency layer is error-detection, not authentication:** the checksum architecture detects random errors ($d = 4$) but cannot resist intentional substitution by an adversary with physical access.
- **No professional audit:** prototype implementations have not undergone independent security audit or formal verification.
- **Usability evidence is preliminary:** timing benchmarks are from the protocol author; no controlled third-party usability study exists for v0.6.0.
- **Manual-fallback substitution detection is weak without RVA:** in a fully electronics-free ceremony where no RVA was recorded, post-recovery substitution detection relies solely on the BIP39 checksum ($\sim 99.6\%$ for 24 words) and operational mitigations such as tamper-evident storage and distributed manifests. This is the weakest substitution-detection profile offered by the protocol; it is a direct consequence of the linear consistency layer's error-detection-only design and the impossibility of computing high-entropy verification tags from GF(2053) arithmetic alone. Deployments that require stronger substitution resistance should use the computational deployment or, at minimum, record an RVA during the sharing ceremony.
- **Manifest compromise is not mitigated by the protocol:** the pre-recovery substitution checks (manifest GIC comparison and audit hash) are conditional on an uncompromised manifest. If the manifest is also tampered with or lost, these checks provide no protection. In inheritance scenarios where the manifest may be the most accessible artifact to an adversary, this dependency deserves explicit operational planning.

9.2 Falsifiability Criteria

F1 (Confidentiality): Formal analysis demonstrates coupled constraints enable $t < k$ shares to narrow search below 2^{256} .

F2 (Manual feasibility): Controlled study ($n \geq 20$) shows $< 50\%$ success rate for manual recovery.

F3 (BIP39 compatibility): Recovered mnemonics fail validation or produce incorrect addresses in major implementations.

F4 (Detection distance): A 3-cell error pattern is found that passes all $r + 4$ check equations.

F5 (Implementation): Security audit identifies a vulnerability with attack complexity $< 2^{128}$.

9.3 Future Work

1. External cryptographic review: seek cryptographic peer review and independent reproduction of the reference test vectors.

2. Controlled usability evidence: gather third-party usability data for the current manual workflow, including success rates, time distributions, and common failure modes.

3. Implementation assurance: pursue independent code review and professional security audit as appropriate to the intended deployment context.

4. Workflow design and sociotechnical analysis: the protocol specifies share arithmetic and verification mechanisms but does not prescribe full sharing or recovery procedures—share distribution logistics, custodian selection, facilitation roles, physical security during multi-hour

manual sessions, rehearsal, rotation, or recovery drills. These operational dimensions are where most real-world secret-sharing failures occur. Future work should develop opinionated deployment profiles (single holder, family, inheritance, assisted workflows) with explicit threat narratives for each, and should address the interaction between ceremony duration, physical exposure, and adversarial opportunity in the manual fallback path.

The present manuscript should therefore be read as an analysis of a promising design point rather than as a claim of deployment readiness.

10 Conclusion

Schiavinato Sharing targets a specific point in the self-custody design space: a BIP39-native threshold-backup format for cold storage and disaster recovery. Its arithmetic layer instantiates standard Shamir sharing directly over 1-indexed BIP39 word indices in $\text{GF}(2053)$, preserving standard BIP39 input and output without custom share alphabets, seed transforms, or permanent dependence on a particular software stack.

The recommended operational path is human-first and software-assisted: software performs arithmetic, validates intermediate state, guides artifact handling, and supports recovery when available and trusted. The continuity property is that recovery remains implementation-independent. A valid threshold set can recover the mnemonic from durable artifacts and modular arithmetic alone, though this manual fallback is slower, operationally heavier, and weaker for substitution detection unless supported by RVA or separately protected manifest material.

The linear consistency layer provides deterministic detection of all 1-, 2-, and 3-cell passive errors, but it is not an authentication mechanism. Substitution resistance is supplied only by layered computational and operational checks: transport hashing, manifest audit hashes when an uncompromised manifest is available, mnemonic-bound BI verification, wallet-bound RVA verification, and artifact separation. These mechanisms improve recovery assurance but remain outside the information-theoretic secrecy claim.

Information-theoretic confidentiality for $t < k$ applies to the unrestricted arithmetic share layer. Reduced Mode is an optional manual-transcription profile that introduces a small, conservatively bounded deviation from exact perfect secrecy in exchange for smaller payloads. Recursive composition is supported for advanced custody topologies, but it is an operationally complex profile rather than the baseline individual-user workflow.

This manuscript is offered as an applied-cryptography design for review, replication, and falsification. The test vectors support independent checking, and prototype implementations are in progress, but the system should not be treated as deployment-ready without further cryptographic review, implementation audit, and usability evidence.

Acknowledgments

The author acknowledges Adi Shamir’s foundational 1979 secret sharing scheme [15], the BIP39 standard [3], and analysis of existing schemes (SLIP39, SSKR, Codex32, SeedXOR) that informed this design. The author thanks Jean Martina, whose review of an earlier version identified the gap between error-detection and adversarial substitution resistance as the protocol’s principal operational risk, and whose feedback directly prompted the Recovery Verification Address design, the explicit “zero substitution detection” statement for the linear consistency layer, and the separation of pre-recovery and post-recovery detection mechanisms in the current version. This work was developed independently without external funding. All errors remain the author’s responsibility.

References

- [1] A. Beimel, “Secret-Sharing Schemes: A Survey,” *Coding and Cryptology, IWCC 2011*, Springer LNCS, vol. 6639, pp. 11–46, 2011.
- [2] P. Wuille, “BIP-32: Hierarchical Deterministic Wallets,” *Bitcoin Improvement Proposals*, 2012. [Online]. Available: <https://bips.dev/32/>.
- [3] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, “BIP-39: Mnemonic code for generating deterministic keys,” *Bitcoin Improvement Proposals*, 2013. [Online]. Available: <https://bips.dev/39/>.
- [4] P. Wuille and G. Maxwell, “BIP-173: Base32 address format for native witness outputs,” *Bitcoin Improvement Proposals*, 2017. [Online]. Available: <https://bips.dev/173/>.
- [5] P. Wuille, “BIP-350: Bech32m format for v1+ witness addresses,” *Bitcoin Improvement Proposals*, 2020. [Online]. Available: <https://bips.dev/350/>.
- [6] G. R. Blundo, A. De Santis, D. R. Stinson, and U. Vaccaro, “Graph decompositions and secret sharing schemes,” *EUROCRYPT ’92*, Springer LNCS, vol. 658, pp. 1–24, 1993.
- [7] V. Buterin, “Why we need wide adoption of social recovery wallets,” Blog post, Jan. 2021. <https://vitalik.ca/general/2021/01/11/recovery.html>.
- [8] A. Poelstra, L. O’Connor, S. Corallo, “BIP-93: codex32: Checksummed SSSS-aware BIP32 seeds,” *Bitcoin Improvement Proposals*, Draft, 2023. [Online]. Available: <https://bips.dev/93/>.
- [9] R. Cramer, I. Damgård, and U. Maurer, “General secure multi-party computation from any linear secret-sharing scheme,” *EUROCRYPT 2000*, Springer LNCS, vol. 1807, pp. 316–334, 2000.
- [10] S. Eskandari, D. Barrera, E. Stobert, and J. Clark, “A First Look at the Usability of Bitcoin Key Management,” *Proc. USEC 2015 (Workshop on Usable Security)*, 2015. doi: <https://doi.org/10.14722/usec.2015.23015>.
- [11] National Institute of Standards and Technology (NIST), *FIPS PUB 180-4: Secure Hash Standard (SHS)*, Aug. 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>.
- [12] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, Feb. 1997. <https://www.rfc-editor.org/rfc/rfc2104>.
- [13] R. Schiavinato Lopez, *Schiavinato Sharing v0.4.1*, GRIFORTIS, 2025. Available at <https://github.com/GRIFORTIS/schiavinato-sharing>.
- [14] Coinkite, “SeedXOR: XOR-based secret sharing for BIP39 mnemonics,” 2021. [Online]. Available: <https://seedxor.com/>.
- [15] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.
- [16] P. Rusnak, A. Kozlik, O. Vejpestek, T. Susanka, M. Palatinus, and J. Hoenicke, “SLIP-0039: Shamir’s Secret-Sharing for Mnemonic Codes,” SatoshiLabs Improvement Proposals, Final, 2019. [Online]. Available: <https://github.com/satoshilabs/slips/blob/master/slip-0039.md>.

- [17] W. McNally and C. Allen, “UR Type Definition for Sharded Secret Key Reconstruction (SSKR),” Blockchain Commons Research, BCR-2020-011, ver. 1.0.1, 2020 (rev. 2021). [Online]. Available: <https://github.com/BlockchainCommons/Research/blob/master/papers/bcr-2020-011-sskr.md>.
- [18] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed. Chapman and Hall/CRC, 2006.
- [19] M. Tompa and H. Woll, “How to share a secret with cheaters,” *Journal of Cryptology*, vol. 1, no. 2, pp. 133–138, 1988.

A Worked Example in GF(13)

We illustrate the full sharing and recovery lifecycle using GF(13) for tractable arithmetic. The real scheme uses GF(2053); the structure is identical. We use the same column tags $\{10, 20, 30\}$ and totals $T_C = 60$ as the real scheme, deferring modular reduction to the evaluation step. This example uses a single row ($r = 1$); with multiple rows, each column checksum sums across all rows in that column.

Setup. Toy wordlist (indices 0–12). Mnemonic: $w_1 = 3, w_2 = 5, w_3 = 7$ (one row, three columns). Row tag $\tau_1^R = 1$, column tags $\tau_1^C = 10, \tau_2^C = 20, \tau_3^C = 30$. Row total $T_R = 1$, column total $T_C = 60$. Scheme: 2-of-3.

A.1 Sharing

Step 1: Word polynomials. Sample $a_1 \in \{0, \dots, 12\}$ for each word:

$$f_{w_1}(x) = 3 + 4x \pmod{13}$$

$$f_{w_2}(x) = 5 + 6x \pmod{13}$$

$$f_{w_3}(x) = 7 + 1x \pmod{13}$$

Evaluate at $x \in \{1, 2, 3\}$ to produce word shares:

x	w_1	w_2	w_3
1	7	11	8
2	11	4	9
3	2	10	10

Step 2: Row checksum and incremental validation.

$$f_{R_1}(x) = (3+5+7+1) + (4+6+1)x = 3 + 11x \pmod{13}$$

Evaluate and validate immediately on each share:

x	w_1	w_2	w_3	R_1	Check: $(w_1+w_2+w_3+1) \pmod{13}$
1	7	11	8	1	$(7+11+8+1) = 27 \pmod{13} = 1 \checkmark$
2	11	4	9	12	$(11+4+9+1) = 25 \pmod{13} = 12 \checkmark$
3	2	10	10	10	$(2+10+10+1) = 23 \pmod{13} = 10 \checkmark$

Step 3: Column checksums.

$$f_{C_1}(x) = (3+10) + 4x = 13 + 4x \pmod{13}$$

$$f_{C_2}(x) = (5+20) + 6x = 25 + 6x \pmod{13}$$

$$f_{C_3}(x) = (7+30) + 1x = 37 + 1x \pmod{13}$$

Evaluate and validate on each share. For share 1:

$$(7+10) \pmod{13} = 4 = C_1, \quad (11+20) \pmod{13} = 5 = C_2, \quad (8+30) \pmod{13} = 12 = C_3.$$

Step 4: GIC. Base: $f_{\text{GIC}}(x) = (3+5+7+1+60) + (4+6+1)x = 76 + 11x \pmod{13}$. Share-bound: $\text{GIC}(x) = (76 + 11x + x) \pmod{13} = (76 + 12x) \pmod{13}$.

Three equivalent validation paths (share 1, GIC = 10):

$$\text{Words: } (7+11+8+1+60+1) \pmod{13} = 88 \pmod{13} = 10 \checkmark$$

$$\text{Rows: } (1+60+1) \pmod{13} = 62 \pmod{13} = 10 \checkmark$$

$$\text{Cols: } (4+5+12+1+1) \pmod{13} = 23 \pmod{13} = 10 \checkmark$$

Step 5: Share assembly. Final share table:

x	w_1	w_2	w_3	R_1	C_1	C_2	C_3	GIC
1	7	11	8	1	4	5	12	10
2	11	4	9	12	8	11	0	9
3	2	10	10	10	12	4	1	8

A.2 Recovery from shares 1 and 3

Step 1: Per-share GIC validation. Share 1: $(7+11+8+1+60+1) \pmod{13} = 88 \pmod{13} = 10 = \text{GIC} \checkmark$. Share 3: $(2+10+10+1+60+3) \pmod{13} = 86 \pmod{13} = 8 = \text{GIC} \checkmark$.

Step 2: Lagrange coefficients and sanity check. $\gamma_1 = 3/(3-1) = 3 \cdot 7 = 8 \pmod{13}$, $\gamma_3 = 1/(1-3) = 1 \cdot (-2)^{-1} = 6 \pmod{13}$.

Sanity: $(8 \cdot 1 + 6 \cdot 3) \pmod{13} = 26 \pmod{13} = 0 \checkmark$.

Step 3: Row-by-row interpolation with incremental validation. Interpolate row 1 words and row checksum, then validate before proceeding:

$$\hat{w}_1 = (8 \cdot 7 + 6 \cdot 2) \pmod{13} = 68 \pmod{13} = 3$$

$$\hat{w}_2 = (8 \cdot 11 + 6 \cdot 10) \pmod{13} = 148 \pmod{13} = 5$$

$$\hat{w}_3 = (8 \cdot 8 + 6 \cdot 10) \pmod{13} = 124 \pmod{13} = 7$$

$$\hat{R}_1 = (8 \cdot 1 + 6 \cdot 10) \pmod{13} = 68 \pmod{13} = 3$$

Incremental check: $(\hat{w}_1 + \hat{w}_2 + \hat{w}_3 + 1) \pmod{13} = (3+5+7+1) \pmod{13} = 3 = \hat{R}_1 \checkmark$. Row passes; proceed.

Step 4: Global validation. Interpolate column checksums and GIC: $\hat{C}_1 = (8 \cdot 4 + 6 \cdot 12) \pmod{13} = 104 \pmod{13} = 0$; $\hat{C}_2 = (8 \cdot 5 + 6 \cdot 4) \pmod{13} = 64 \pmod{13} = 12$; $\hat{C}_3 = (8 \cdot 12 + 6 \cdot 1) \pmod{13} = 102 \pmod{13} = 11$.

Column checks: $(3+10) \pmod{13} = 0 = \hat{C}_1 \checkmark$; $(5+20) \pmod{13} = 12 = \hat{C}_2 \checkmark$; $(7+30) \pmod{13} = 11 = \hat{C}_3 \checkmark$.

Interpolate GIC: $\hat{\text{GIC}} = (8 \cdot 10 + 6 \cdot 8) \pmod{13} = 128 \pmod{13} = 11$. The $+x$ terms cancel automatically.

Three validation paths:

$$\text{Words: } (3+5+7+1+60) \pmod{13} = 76 \pmod{13} = 11 = \hat{\text{GIC}} \checkmark$$

$$\text{Rows: } (3+60) \pmod{13} = 63 \pmod{13} = 11 = \hat{\text{GIC}} \checkmark$$

$$\text{Cols: } (0+12+11+1) \pmod{13} = 24 \pmod{13} = 11 = \hat{\text{GIC}} \checkmark$$

Step 5: Output. Recovered mnemonic: $(w_1, w_2, w_3) = (3, 5, 7)$. Convert to wordlist entries and validate the checksum (in the real scheme, BIP39 validation).

B Digital Payload Encoding (Reference)

This appendix provides the reference byte layout for the digital envelope. The canonical digital object is the Core Payload bytes; QR encodings prepend a self-identifying ASCII prefix. Supplementary implementation artifacts are available at <https://github.com/GRIFORTIS/schiavinato-sharing>. Row and column checksums are computed from the share data for human-readable display but are not serialized in the digital envelope. The only arithmetic check value serialized in the envelope is the printed GIC. Transport hashes and manifest Audit Hashes therefore commit to the serialized word-share values and GIC, not to separately printed row or column checksum cells; those cells are verified by recomputation from the serialized or transcribed share data.

B.1 Shared Header Fields

The protocol version byte is 0x01. The flags byte has the following bit layout:

Bits	Meaning	Encoding
0–2	Word count	0=12, 1=15, 2=18, 3=21, 4=24
3–4	Nesting depth	Full: 0–3 for 1–4 active layers; Reduced: bit 3 only, bit 4 MUST be 0
5–7	Wallet/derivation hint	000 Generic, 001 BIP84, 010 BIP86, 011 BIP49, 100 BIP44, 101 EVM, 110 hardware default, 111 reserved

The wallet/derivation hint is a coarse recovery aid only. It does not replace the share-local wallet-context hint needed for RVA verification.

Threshold and share-index packing. Full Mode allocates two bytes each for threshold k and share index x . Reduced Mode allocates one byte each. Let depth 0 denote one active layer, depth 1 two active layers, and so on. In Full Mode, depth 0 stores layer 0 in byte 0 and requires byte 1 to be zero; depth 1 stores layer 0 in byte 0 and layer 1 in byte 1; depth 2 stores layer 0 in the low nibble of byte 0, layer 1 in the high nibble of byte 0, layer 2 in the low nibble of byte 1, and requires the high nibble of byte 1 to be zero; depth 3 stores layers 0–3 in the four nibbles in that same order. In Reduced Mode, depth 0 stores layer 0 in the single byte; depth 1 stores layer 0 in the low nibble and layer 1 in the high nibble. All share indices are nonzero, and per-layer recovery indices must be distinct.

B.2 Full Mode Core Payload

$$\text{CorePayload} = \text{Header} \parallel B \parallel I \parallel \text{ShareData} \parallel H$$

Offset	Length	Field
0	1	Protocol version (0x01)
1	1	Flags
2–3	2	Threshold k (layer-dependent packing)
4–5	2	Share index x (layer-dependent packing)
6–13	8	Session Batch ID B
14–25	12	Blinded Identity I
26	variable	Share data (words + GIC, 12-bit packed)
26 + share_data_len	16	Transport Hash H

Share data encodes $w_1[x], \dots, w_\ell[x]$ followed by $\text{GIC}[x]$ as 12-bit unsigned integers, most-significant bit first, padded with zero bits to the next full byte. Full Mode core length is $42 + \text{share_data_len}$ bytes.

ℓ	Elements	Share data bits	Pad bits	Core bytes / QR bytes
12	13	156	4	62 / 66
15	16	192	0	66 / 70
18	19	228	4	71 / 75
21	22	264	0	75 / 79
24	25	300	4	80 / 84

The Transport Hash is the first 16 bytes of SHA256 over all preceding Core Payload bytes (header, Session Batch ID, Blinded Identity, and Share Data). A mismatch on decode is a STOP condition. Full Mode QR bytes are ASCII SCHI (0x53 0x43 0x48 0x49) followed by the Core Payload. The baseline QR is V5 (37×37), error correction level M, whose byte-mode capacity is 84 bytes.

B.3 Reduced Mode Core Payload

$$\text{CorePayload} = \text{Header} \parallel B \parallel I \parallel \text{ShareData}$$

Offset	Length	Field
0	1	Protocol version (0x01)
1	1	Flags
2	1	Threshold k
3	1	Share index x
4–7	4	Session Batch ID B
8–15	8	Blinded Identity I
16	variable	Share data (words 11-bit + GIC 12-bit, packed)

Reduced Mode has no Transport Hash. Transport integrity relies on QR error correction plus GIC validation. Word values are encoded as 11-bit unsigned integers; the printed GIC remains a 12-bit value over GF(2053). Values are concatenated most-significant bit first and padded with zero bits to the next full byte. During sharing, any word polynomial whose evaluation at any share index produces a value greater than 2047 is rejected and regenerated; the printed GIC is not range-restricted. Reduced Mode core length is 16 + `share_data_len` bytes.

ℓ	Word bits	GIC bits	Total bits	Pad bits	Core bytes / QR bytes
12	132	12	144	0	34 / 36
15	165	12	177	7	39 / 41
18	198	12	210	6	43 / 45
21	231	12	243	5	47 / 49
24	264	12	276	4	51 / 53

Reduced Mode QR bytes are ASCII SC (0x53 0x43) followed by the Core Payload. The baseline QR is V3 (29×29): error correction level M for 12- and 15-word mnemonics, and level L for 18-, 21-, and 24-word mnemonics.

B.4 QR Prefix Decode Rule

Decoders MUST inspect QR bytes before parsing. If the input starts with ASCII SCHI, strip the 4-byte prefix and parse the remainder as Full Mode. If the input starts with ASCII SC and does not continue with HI, strip the 2-byte prefix and parse the remainder as Reduced Mode. Implementations MAY accept raw Core Payload bytes without a QR prefix in text-import or legacy contexts, but then MUST infer the mode from field structure and total length and still enforce all validation rules.

C Pre-Computed Lagrange Coefficients

Lagrange coefficients are determined entirely by share indices and contain zero secret information. They may be computed on any device (even untrusted) or verified against multiple sources.

Table 4: Lagrange coefficients in GF(2053)

Scheme	Shares	Coefficients (γ)
2-of-3	{1, 2}	(2, 2052)
	{1, 3}	(1028, 1026)
	{2, 3}	(3, 2051)
2-of-4	{1, 2}	(2, 2052)
	{1, 3}	(1028, 1026)
	{1, 4}	(1370, 684)
	{2, 3}	(3, 2051)
	{2, 4}	(2, 2052)
	{3, 4}	(4, 2050)
3-of-5	{1, 2, 3}	(3, 2050, 1)
	{1, 2, 4}	(687, 2051, 1369)
	{1, 2, 5}	(1029, 1367, 1711)
	{1, 3, 4}	(2, 2051, 1)
	{1, 3, 5}	(1285, 512, 257)
	{1, 4, 5}	(686, 1367, 1)
	{2, 3, 4}	(6, 2045, 3)
	{2, 3, 5}	(5, 2048, 1)
	{2, 4, 5}	(1372, 2048, 687)
	{3, 4, 5}	(10, 2038, 6)

Computing coefficients for other share sets. For any recovery set $S = \{x_1, \dots, x_k\}$ of distinct nonzero share indices, the interpolation coefficient for share x_j at the origin is

$$\gamma_j = \prod_{\substack{i=1 \\ i \neq j}}^k \frac{x_i}{x_i - x_j} \pmod{2053}.$$

All divisions are field divisions in GF(2053), implemented by multiplying with modular inverses. The recovered secret is then

$$f(0) = \sum_{j=1}^k \gamma_j y_j \pmod{2053},$$

where y_j is the share value at index x_j . Because the coefficients depend only on the public share indices, they may be precomputed once for any valid threshold set and reused across all rows, checksums, and GIC interpolation steps.

D Deployment and Artifact Trust Boundaries

This appendix summarizes the deployment model at the level of artifact classes and artifact-production tiers rather than application screens. Secret-bearing artifacts remain inside the trusted secret-handling boundary. Encrypted sensitive resume artifacts may move to a Companion device as ciphertext only. Manifest artifacts contain no plaintext shares, but remain operationally sensitive and are stored separately from shares.

D.1 Printer and Output Tiers

The software-assisted sharing workflow separates computation trust from output-device trust. Four output tiers are defined:

1. **Tier 1—offline non-network printer:** a USB-only or otherwise non-networked printer directly attached to the air-gapped ceremony device. Secret-bearing material may be printed. Full Mode is recommended.
2. **Tier 2—personal network-capable printer:** a printer controlled by the user but capable of Wi-Fi, Ethernet, cloud print, stored jobs, or similar features. Secret-bearing material **MUST NOT** be printed. Manifest metadata and blank share templates may be printed, accepting that the metadata is operationally sensitive.
3. **Tier 3—untrusted printer:** workplace, library, print-shop, shared, or otherwise uncontrolled printers. Only blank structure may be printed: grids, labels, checkboxes, empty fields, and optional blank QR overlays. No Session Batch ID, Blinded Identity, Printed GIC, Audit QR, or secret-bearing data should be sent to this tier.
4. **Tier 4—no printer:** all output is hand-transcribed from the air-gapped device display.

For Tiers 2–4, secret-bearing share values, row checksums, column checksums, printed GIC, and payload QR/text exports are copied by hand from the trusted device and then verified either by software-assisted re-entry/scanning or by manual arithmetic checks. Reduced Mode is recommended when QR transcription burden dominates, but Full Mode remains available when the operator accepts the larger QR grid in exchange for the Transport Hash.

Artifact class	Examples	Allowed handling	Lifecycle
Plaintext secret-bearing	Mnemonic, plaintext share tables, payload QR/text, plaintext coefficients	Air-gapped ceremony device and Tier 1 trusted non-networked printer only; never sent to Companion or network/public printers	Stored only as distributed shares; working copies destroyed
Encrypted sensitive	Digital Resume payload, encrypted coefficient pool, encrypted Single-Entropy QR	May be stored by Companion as ciphertext only; Encryption Passphrase and decrypted contents remain on the air-gapped device	Temporary ceremony-continuity material; deleted after successful share creation
Operational metadata	Manifest, Batch ID, Blinded Identity, Printed GIC map, Audit Hashes, blank templates	May be printed on Tier 1 or Tier 2 according to printer-tier rules; Tier 3 receives only blank structure	Stored separately from shares for audit/inventory
Recovery inputs	k shares and optional manifest context	Recovered on paper or trusted air-gapped recovery device; manifest assists audit and context but is not required for arithmetic recovery	Used only during recovery or audit drill

Table 5: Deployment and artifact trust boundaries. Secret-bearing material remains within trusted secret-handling components. Encrypted sensitive resume artifacts may leave the air-gapped device only as ciphertext. Manifest metadata contains no plaintext shares, but remains operationally sensitive and is stored separately from shares.